



T.C. MARMARA ÜNİVERSİTESİ
TEKNOLOJİ FAKÜLTESİ
ELEKTRİK-ELEKTRONİK MÜHENDİSLİĞİ BÖLÜMÜ

BLM1030: BİLGİSAYAR PROGRAMLAMA 2025-2026 BAHAR DÖNEMİ PROJE ÖDEVİ

Proje Adı: Araç Kiralama Uygulaması

Geliştirici Ekip: Ömer Faruk Boyraz, Ömer Salim Aydoğan, Egemen Eyüboğlu

Ders: BLM1030 Bilgisayar Programlama

Tarih: 28 Mayıs 2026

Projenin Amacı ve Çözdüğü Gerçek Dünya Problemi

Gerçek Dünya Problemi

Araç kiralama firmalarının envanter, müşteri, kiralama, bakım ve ödeme takibini manuel veya e-tablolarla yapması; veri kaybına, çift rezervasyonlara, geciken araç bakımlarına ve güvenlik açıklarına yol açmaktadır.

Doldurduğu Boşluk ve Önem

Geliştirilen **Araç Kiralama Otomasyonu**, müşteri ve yönetici panellerini tek çatı altında toplayarak tüm süreçleri dijitalleştirir. SQLite veritabanı desteği ve kullanıcı dostu grafik arayüzüyle güvenli, hatasız ve hızlı bir yönetim sağlar.

Mimari ve Tasarım (UML Sınıf Diyagramı)

Sistemdeki sınıfların yapıları, işlevleri ve ilişkileri hem kod düzeyinde (Mermaid UML) hem de görsel sınıf şeması olarak aşağıda sunulmuştur.

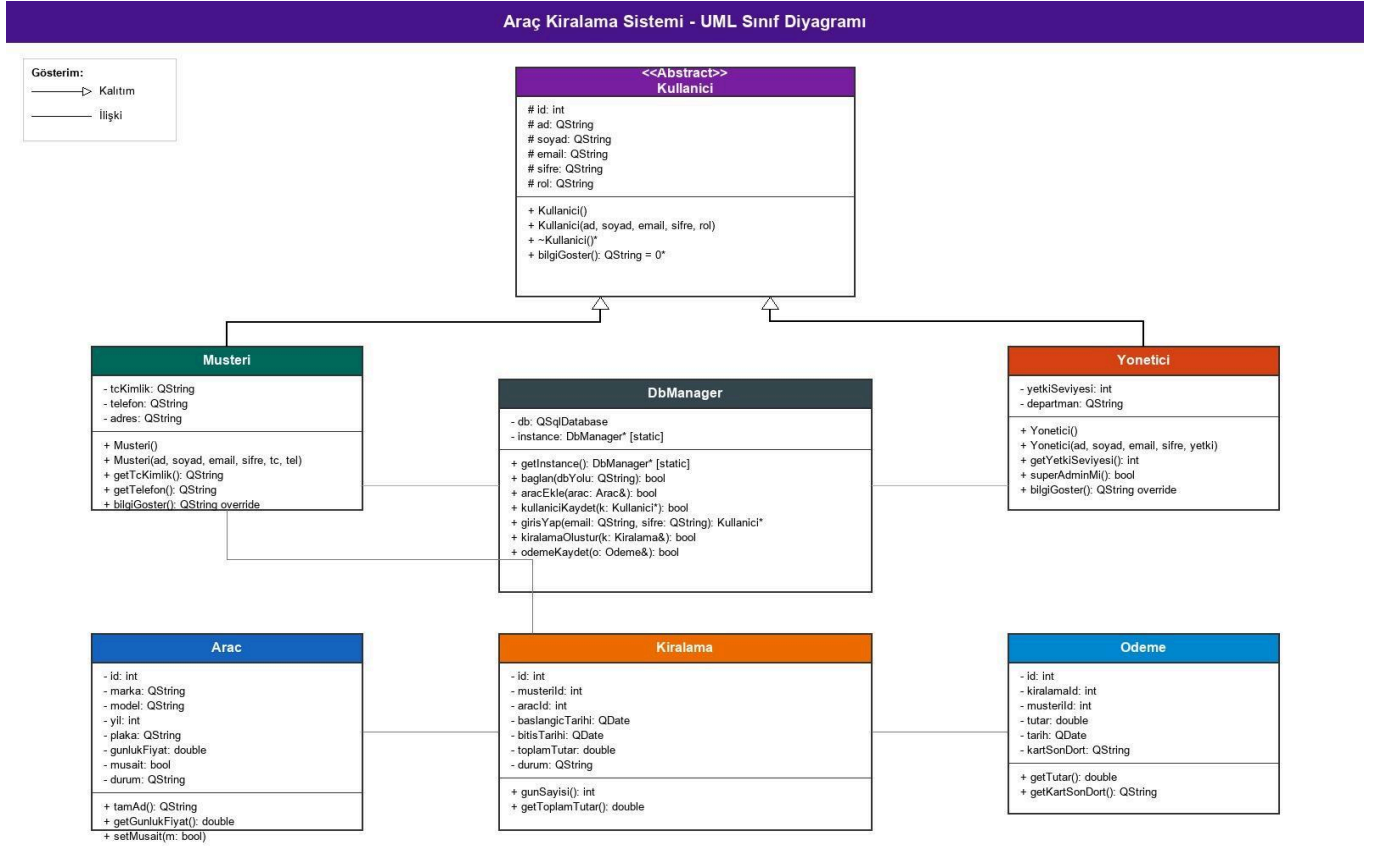
Kod Düzeyinde UML Sınıf Şeması (Fonksiyonlu Yapı)

```
classDiagram
    Kullanici <|-- Musteri : Kalıtım
    Kullanici <|-- Yoneticisi : Kalıtım
    DbManager ..> Kullanici : Yönetim
    DbManager ..> Arac : Envanter
    DbManager ..> Kiralama : Kiralama
```

DbManager ..> Odeme : Ödeme

```
class Kullanici {
    <<Abstract>>
    #int id
    #QString ad, soyad, email, sifre, rol
    +bilgiGoster() QString*
}
class Musteri {
    -QString tcKimlik, telefon, adres
    +bilgiGoster() QString
}
class Yonetici {
    -int yetkiSeviyesi
    -QString departman
    +bilgiGoster() QString
    +superAdminMi() bool
}
class Arac {
    -int id, yil, km
    -QString marka, model, plaka, renk, kategori, durum
    -bool musait
    -double gunlukFiyat
    +tamAd() QString
}
class Kiralama {
    -int id, musterId, aracId
    -QDate baslangicTarihi, bitisTarihi
    -double toplamTutar
    -QString durum, notlar
    +gunSayisi() int
}
class Odeme {
    -int id, kiralamaId, musterId
    -double tutar
    -QDate tarih
    -QString durum, kartSonDort
}
class DbManager {
    -QSqlDatabase db
    -static DbManager* instance
    +static getInstance() DbManager*
    +baglan(dbYolu) bool
    +aracEkle(Arac&) bool
    +kullaniciKaydet(Kullanici*) bool
    +girisYap(email, sifre) Kullanici*
}
```

Görsel Sınıf Şeması (UML Diyagramı)



Şekil 1: Araç Kiralama Otomasyonu UML Sınıf Şeması

Nesne Yönelimli Programlama (OOP) Prensiplerinin Uygulanması

Projede kullanılan nesne yönelimli programlama prensipleri ve ilgili kod örnekleri aşağıda özetlenmiştir:

Soyutlama (Abstraction)

Gereksiz detayları gizleyip sadece ortak arayüzü sunmaktır. **Kullanici** sınıfını saf sanal **bilgiGoster()** metoduyla tanımlayarak soyut bir taban sınıf haline getirdik ve alt sınıfların bu metodu ezmesini zorunlu kıldık.

```
// model/kullanici.h
class Kullanici {
protected:
    int id; QString ad; QString soyad; QString email; QString sifre; QString rol;
public:
    Kullanici(QString ad, QString soyad, QString email, QString sifre, QString rol);
    virtual ~Kullanici();
    virtual QString bilgiGoster() const = 0; // Saf sanal metot
};
```

Sarmalama (Encapsulation)

Sınıf verilerini gizleyip verilere kontrollü getter/setter metotlarıyla erişilmesini sağlamaktır. **Arac** sınıfında nitelikler private tanımlanmış, negatif fiyat veya km girişi engellenmiştir.

```
// model/arac.h
class Arac {
private:
    int id; QString marka; double gunlukFiyat; int km;
public:
    double getGunlukFiyat() const { return gunlukFiyat; }
    void setGunlukFiyat(double f) { if(f > 0) gunlukFiyat = f; } // Kontrol
    void setKm(int k) { if(k >= 0) km = k; }
};
```

Kalıtım (Inheritance)

Ortak özellikleri taban sınıftan miras alarak kod tekrarını önlemektir. **Musteri** ve **Yonetici** sınıfları ortak niteliklerini (ad, soyad, email, sifre) **Kullanici** sınıfından miras almaktadır.

```
// model/musteri.h
class Musteri : public Kullanici {
private:
    QString tcKimlik; QString telefon;
public:
    Musteri(QString ad, QString soyad, QString email, QString sifre, QString tc, QString tel);
    QString bilgiGoster() const override;
};
```

3.4. Çok Biçimlilik (Polymorphism)

Aynı isimdeki bir metodun farklı nesnelerde farklı davranabilmesidir. **bilgiGoster()** metodu **Musteri** ve **Yonetici** sınıflarında rollerine uygun verileri gösterecek şekilde ezilmiştir (override). Ayrıca giriş yapan kullanıcının türü çalışma zamanında **dynamic_cast** ile tespit edilmektedir.

```
// musterii.cpp & yonetici.cpp
QString Musteri::bilgiGoster() const {
    return QString("MUSTERI: %1 %2 | TC: %3").arg(ad).arg(soyad).arg(tcKimlik);
}

// loginwindow.cpp
Kullanici* k = DbManager::getInstance()->girisYap(email, sifre);
if (k->getRol() == "yonetici") {
    Yonetici* y = dynamic_cast<Yonetici*>(k); // Downcasting
    // ...
}
```

Grafik Kullanıcı Arayüzü (GUI) ve Backend Etkileşimi

Signals & Slots (Sinyal ve Yuva) Mekanizması

Qt'nin olay tabanlı yapısı arayüz ile backend mantığını bağlar. Giriş butonuna tıklandığında otomatik olarak slot metodu tetiklenir.

```
void LoginWindow::on_btnGiris_clicked() {
    Kullanici* k = DbManager::getInstance()->girisYap(ui->txtEmail->text(),
    ui->txtSifre->text());
    // ...
}
```

Veri Doğrulama ve Regex Kullanımı

Giriş hatalarını ve SQL açıklarını engellemek için düzenli ifadeler (Regex) kullanılmıştır. TC Kimlik 11 hane, telefon numarası ise 10 hane olarak kısıtlanmış ve Qt Validator'lar ile kontrol edilmiştir.

Veritabanı Mimarisi ve Singleton Tasarım Deseni

Singleton (Tekil) Tasarım Deseni

Veritabanı bağlantısının tek bir açık nesne üzerinden yönetilmesi ve kaynakların verimli kullanılması için **DbManager** sınıfında **Singleton Tasarım Deseni** uygulanmıştır.

```
// database/dbmanager.cpp
DbManager* DbManager::instance = nullptr;
DbManager* DbManager::getInstance() {
    if (!instance) instance = new DbManager();
    return instance;
}
```

Veritabanı Tabloları (Şema Tasarımı)

SQLite üzerinde yapılandırılan tablolar şunlardır:

- * **araclar:** Araç envanteri ve müsaitlik durumları.
- * **kullaniciilar:** Müşteri ve yönetici kayıtları.
- * **kiralamalar:** Tarih, ücret ve kiralama durum bilgileri.
- * **bakim_kayitlari:** Bakımdaki araçların maliyet ve arıza kayıtları.
- * **kartlar ve odemeler:** Müşteri kart verileri ve kiralama ödeme geçmişi.

Projenin İlave İşlevleri ve Özellikleri

1. **Dinamik Bakım Sistemi:** Bakıma alınan araç 'Müsait Değil' durumuna geçer ve bakımdan çıkarılana kadar kiralanamaz.
2. **Ödeme Arayüzü ve Kart Maskeleye:** Kayıtlı kartlar ödeme ekranında maskelenerek (**** *
**** 1234) gösterilir.
3. **Gerçek Zamanlı Filtreleme:** Arama çubukları, listeleri SQL sorgusu atmadan anlık olarak filtreler.

Ekip Çalışması ve İş Bölümü

Ömer Faruk Boyraz: Backend, SQLite veritabanı sorguları, Singleton tasarımı ve OOP veri modellerinin kodlanması.

Ömer Salim Aydoğan: Qt Designer ile giriş, ana pencere ve yönetim panellerinin UI/UX tasarımları.

Egemen Eyüboğlu: Signals & Slots bağlantıları, Regex veri doğrulamaları, kart maskeleme mantığı ve panel geçişleri.

Çalıştırma ve Demo Adımları

1. Projei Qt Creator ile açıp derleyin.
2. Yönetici girişi: Email: admin@rentacar.com / Şifre: [admin123](#)
3. Müşteri girişi: Kayıt ol panelini kullanarak üye olun ve giriş yapın.

Sonuç ve Kazanımlar

Bu proje ile nesne yönelimli programlama prensipleri pekiştirilmiş, Qt ile olay tabanlı GUI geliştirme, ilişkisel veritabanı entegrasyonu ve ekip çalışması pratik edilmiştir.